

NOTAS PRÁCTICO ICS 4K1 - 2025

Clases grabadas:

<https://docs.google.com/spreadsheets/d/1f-zL7dxwD5cmVFyBrRYh79dEnoah73zg/edit?gid=742131616#gid=742131616>

https://docs.google.com/spreadsheets/d/11O49jPabffwwG9PcXZGAhBUwPZ_Dkhqu/edit?gid=743353583#gid=743353583

Clases de equivalencia:

https://docs.google.com/document/d/13Ae53TAs7s-R3Sbvg_41hXlc1_XZl6w4lyluSi7aHk0/edit?tab=t.0

Caso de prueba:

<https://docs.google.com/spreadsheets/d/1QeH7oDbUrVmUImuo36CCNm-aCZqw9fm-O1LTvry76lc/edit?gid=330830909#gid=330830909>

Imprimir los templates por las dudas.

Una clase de equivalencia para una US.

Uno o dos casos de prueba.

Testing de caja negra

Para una User Story dada.

Ejemplo US: "Ver mapa de taxis".

- Sí se incluye en la US: "Pasando el mouse sobre el ícono".
- No se incluye en la US: "Selección del taxi".

Basado en las especificaciones:

a. Método de partición de equivalencias:

Determinar para una funcionalidad dada cuáles son todas las condiciones externas que participan.

Luego para cada una de esas condiciones externas analizar cuáles son los subconjuntos de valores posibles que pueden tomar y que producen un resultado equivalente.

b. Método de valores límites:

En vez de seleccionar cualquier elemento como representativo de una clase de equivalencia se seleccionan los bordes de una clase.

La mayor cantidad de defectos se encuentra en los extremos de los intervalos o puntos límites.

La exactitud es clave.

Sirve entre otras cosas para encontrar duplicados.

Ejemplo: >, >=, <, <=, y, o, etc.

- Ingresar con 18 años: Límite inferior de clase de equivalencia válida.
- Ingresar con 100 años: Límite superior de clase de equivalencia válida.
- Ingresar con 0 años: Límite inferior de clase de equivalencia inválida.
- Ingresar con 17 años: Límite superior de clase de equivalencia inválida.

Basado en la experiencia:

a. Adivinanza de defecto.

b. Testing exploratorio.

Condiciones externas

Todas las variables en juego o condiciones que van a guiar la ejecución de la funcionalidad.

Entradas y salidas posibles.

NO es la interfaz visual. Tratar de abstraerse de eso.

Pueden ser:

- Rango de valores continuos.
- Valores discretos.
- Selección simple.
- Selección múltiple.

Ejemplo: números, strings, fechas, archivos, fotos, coordenadas, etc.

Condiciones externas de entrada:

Si hay validaciones con respecto a otra condición externa, aclarar que tienen relación en las clases de equivalencia si lo ven necesario, sino no hace falta.

Ejemplo si selecciono una opción: “edad ocho años si selecciona x cosa y mayor a cero si selecciona x cosa”.

Usuario logueado:

La condición externa “Usuario logueado” va SÓLO si es relevante.

En el caso del taxi era importante porque sólo un administrador tiene el poder de ejecutar la US.

Condición externa tarjeta de crédito y DNI:

En el DNI la salida es la misma.

En tarjeta de crédito 7 u 8 dígitos con distintas clases de equivalencia porque las salidas son distintas.

Dependiendo cuántos caracteres de ingreso se valida si es correcto o no dependiendo de la marca, fecha de vencimiento, etc.

Precondición: La tarjeta tiene que tener x condiciones (ej vencida, aprobada, sin fondos, etc. Para probar lo que va a pasar con cada tipo)

Nos conectamos con mercado pago y la tarjeta es nombre etc y está aprobada, con fondos, no vencida, etc.

Clases de equivalencia

Subconjunto de valores que puede tomar una condición externa para el cual si yo tomo cualquier miembro de ese subconjunto, el resultado en la ejecución de la funcionalidad es equivalente (NO igual).

Ejemplo de subconjunto de valores:

- Edad ≥ 18 -> Puede comprar. Sin importar qué edad sea.
- Edad < 18 -> No puede comprar. Sin importar qué edad sea.
- En estos casos, 17 y 18 son clases de una misma condición externa.

Cantidad de clases de equivalencia:

Para una condición externa puedo tener una o muchas clases de equivalencia válidas y/o inválidas.

La cantidad depende mucho del mensaje de error que esperamos del software.

- Misma clase de equivalencia:
Si la salida es equivalente y el procesamiento es el mismo.
- Diferentes clases de equivalencia si:
Si las salidas son diferentes.
Si el procesamiento es diferente (por más de que la salida sea la misma).
Ejemplo: Cierra lunes y feriados (Eco park).

Dentro de una misma condición de entrada, cuando hay dos clases de equivalencia (se divide), hay que hacer para cada una de ellas la clase inválida? O se hacen juntas para la condición de entrada? No siempre es necesario una para cada una, puede ser en conjunto.

Cómo determinar todo el conjunto de clases de equivalencia de una condición externa:

Las clases de equivalencia válidas e inválidas de una misma condición externa son mutuamente excluyentes entre sí (teoría de conjuntos).

La clase de equivalencia inválida "Formato inválido" SOLO VA si anteriormente en la clase de equivalencia válida aclaré cuál es el formato válido.

Algunas clases de equivalencia se pueden representar en precondiciones.

Dependencia:

Cuando hay dependencia en las clases de equivalencia.

En la primera clase dividimos en dos clases de equivalencia distintas porque tienen distintas salidas.

En la clase de equivalencia de la cual depende, también hay que dividirla según las otras dos?

Está bien de las dos formas. Dependencia que no sea un mensaje de error.

Dependencias.

Errores de las clases de salidas: con rangos y dentro de un rango se permite otro rango y otro rango. ¿En los errores hace falta aclarar?

Depende del sentido del software. Ejemplo edad que no corresponde para cierta actividad SI ACLARO en el mensaje de error distinto de la edad no debe ser cero.

En el parcial no más de cinco.

Qué coincidan con las clases de equivalencia inválidas.

Un error de salida puede llegar a juntar dos clases de equivalencia distintas.

Dependencias:

Se tiene que dividir necesariamente dependencia y dependencia?

Tienen salidas distintas y se dividen en dos. Clases distintas según lo que haya elegido.

Mensajes de error y clases de equivalencias inválidas:

Ver dependiendo el caso si es mejor distintos mensajes de error o uno solo.

Si es uno solo con la clase de equivalencia invalida "Cualquier otra cosa no válida" es suficiente.

Mail y correo:

Clase de equivalencia válida de envío de mail: Se envía mail con.. (aclarar TODO).

No existe clase de equivalencia inválida de envío de mail. Quedan vacías.

Medio de pago tarjeta de crédito: Me obliga a ingresar más datos.

Fechas:

Clases de equivalencia por ejemplo con rango.

Caso de prueba deben ser todos datos exactos.

Clases de equivalencia de entrada

Entradas		
Condición externa	Clases de equivalencia válidas	Clases de equivalencia inválidas

Clases de equivalencia de salida

Todo lo que muestra el software.

A partir del software pueden ser generadas.

Lo que se muestre ya debe estar en el software o fue cargado previamente.

Ejemplo: mapa, datos del taxi, colores, todos los mensajes de error, etc.

Los cálculos se ponen como salida y diferente si se muestra en el backend o en el front.

Si la condición externa es "Error":

¿Cada mensaje de error es una condición externa aparte? NO.

Tengo una sola condición externa "Error" o "Mensaje" y múltiples clases de equivalencia válidas e inválidas.

Explicitar exactamente QUÉ dice el error: "Se muestra un mensaje de error con los datos de..." o "Error, la chapa tiene un formato inválido".

Mínimo cinco mensajes de error en el parcial.

Salidas		
Condición externa	Clases de equivalencia	Clases de equivalencia

	válidas	inválidas

Casos de prueba (CP)

Una vez identificadas todas las condiciones externas y clases de equivalencia.

Poner las clases de equivalencia en uso.

Busco poder probar la mayor cantidad de clases de equivalencia con la menor cantidad de casos de prueba.

Tomo de cada condición externa una clase de equivalencia en particular y elijo un valor representativo para poder conformar mi caso de prueba.

Receta.

Conjunto de pasos ordenados que el tester debe seguir para ejecutar una funcionalidad con una especificación de cuáles van a ser los valores que va a ingresar.

Deben ser valores miembros de alguna clase de equivalencia de esa condición externa.

Un mismo caso de prueba puede agrupar dos o más clases de equivalencia.

ID del caso de prueba.
Prioridad (Alta, Media, Baja).
Nombre del caso de prueba.
Clases de equivalencia.
Precondiciones: 1. Precondición nro 1 2. Precondición nro ... 3. Precondición nro n
Pasos: 1. Paso nro 1 2. Paso nro ... 3. Paso nro n
Resultado esperado.

ID del caso de prueba

Número que identifica inequívocamente este artefacto.

Prioridad

Relacionada con qué CP voy a ejecutar primero.

Es el orden de la ejecución de los CP para encontrar defectos.

Si tengo que seleccionar entre varios CP para ejecutar voy a priorizar aquellos que me permitan encontrar la mayor cantidad de defectos y que además garanticen la salud de la funcionalidad.

- Alta: Siempre están acá los escenarios que producen caminos felices.
- Baja: CP de validaciones, defectos de menor cantidad de frecuencia, formatos, valores, etc.
- Media: Todo lo intermedio.

Nombre del caso de prueba

Describir de forma clara cuál es el escenario que estamos probando.

Lo más distinguido posible sin ser demasiado largo.

Aclarar si es exitosa (pasa) o si no.

Ejemplo: "Ingresar de forma exitosa con edad mayor a 18 años".

Clases de equivalencia

Las que se validan en el caso de prueba.

Precondiciones

Todo lo que debemos tener de información para poder probarlo.

Todos los casos de prueba deben ser 100% reproducibles.

Todo lo necesario para poder reproducirlo.

Cosas exactas.

Cosas de BDA, artículos, nombre, precio, cosas pre-cargadas, etc.

Si tengo la precondición "Usuario "Salva" logueado con permisos de administrador" entonces no necesito un paso nro 1 que sea "El usuario "Salva" se loguea en el sistema".

Algunas clases de equivalencia se pueden representar en precondiciones.

Hora en la que se carga algo es una precondición?:

Sí, de fecha y hora. Se asume que toma la fecha y la hora de cuando terminé siendo un tiempo razonable para el proceso (ej registrar actividad, 5 segundos. Si es uno que siempre tarda una hora le sumo a la fecha y hora de inicio del registro una hora que sé qué es lo que demora)

Precondición: el proceso suele durar x tiempo.

Muestra fecha y hora actual + x tiempo aproximado

Si la hora actual o la fecha actual se usará para un cálculo podría ser una condición externa y sigue siendo precondición? Solo si la fecha como tal tiene ingerencia sobre lo que estoy haciendo. Es raro porque normalmente lo uso como validador o comparador de otra condición externa y ahí es precondición.

Precondición vs condiciones externas:

Ejemplo: usuario logueado o ubicación activada.

Paso: todo lo que tenga que hacer en la ejecución del caso de prueba es un paso y debe estar como paso.

Precondición: todo lo que tiene que estar desde antes pre-configurado es una precondición.

Ejemplo al registrar actividad no me interesa probar si está la sesión iniciada, no me hace falta ni que esté la US de iniciar sesión, puedo tener todo hardcodeado y probar igual, no estoy probando el inicio de sesión.

Precondición usuario logueado:

Cómo precondición va a estar en la mayoría de los pasos.

Puede pasar que el usuario logueado puede ser precondición y paso (ejemplo taxi) en caso de que el perfil importe.

Pasos

Conjunto de operaciones ordenadas y numeradas.

Claro, sin ambigüedades.

Los debe ejecutar el tester para conseguir el resultado esperado.

Ejemplo: El "Rol" selecciona la opción "Ingresar".

Subida de archivos:

Ejemplo: imagen.png está en la computadora (precondición), selecciona imagen.png (paso).

Si tiene que tener peso específico u otra característica se aclara.

Resultado esperado

Asociado al paso que me interesa.

Del escenario que estoy ejecutando.

Puede haber más de uno.

Ejemplo: Se muestra el mensaje "Bienvenido al sitio web".

Campo nombre y lo pongo mal. Rta mensaje nro que le asocio es 3 o 4? Lo asocio al paso 7 que nos da la salida esa. Es una asociación a este paso, no es lo que le sigue.

Precondiciones, pasos y resultado esperado deben especificar sí o sí valores identificables para cada una de las clases de equivalencias para garantizar la reproducibilidad del CP. Son un valor concreto.

No se ingresa un valor:

Si tengo más de un valor de entrada voy a poder hacer un escenario completo que sea "No sé ingresa ningún valor" o "No sé ingresan los campos obligatorios".

En el CP me encargo de aclarar qué sí se ingresó y que no.

El resultado esperado en este caso puede ser:

- Un solo mensaje:
"Debe completar todos los datos obligatorios".
- Un conjunto de mensajes:
El sistema muestra el mensaje: "Debe ingresar una edad".
El sistema muestra el mensaje: "Debe ingresar un apellido".
Así para cada campo no ingresado.

No hace falta tener un CP para cada campo que puedo llegar a no ingresar.

Ejemplo: Tengo muchas condiciones externas (nombre, apellido, edad, etc.).

Cada una de ellas tiene una clase de equivalencia inválida de: "No ingresa valor".

Por lo tanto puedo tener un solo CP que agrupe las tres condiciones externas y para cada una voy a usar una clase de equivalencia.

Ejemplo de CP: "Registrar cuenta de usuario sin ingresar valores".

Varias salidas:

Si en un CP tengo varias salidas o pantallas lo puedo especificar en el resultado esperado.

El sistema puede mostrar varias cosas juntas pero debe quedar claro exactamente qué.

Precondición vs Condición externa:

Ejemplo tener que tener reserva para entrar a un bar.

Es una precondición si está asociada la reserva a mi cuenta de usuario y debo tenerla.

No es una condición externa que yo pueda ponerle a la funcionalidad.

Depende de que el usuario logueado tenga una reserva anteriormente.

Es una condición externa de entrada si yo tengo que ingresar edad y número de reserva.

“Maximizar las clases de equivalencia inválidas”:

Formulario que se valida a medida que escribo -> Respuesta esperada: todos distintos mensajes de error, cada uno con su paso asociado.

Formulario que hago click al final y valida todo junto -> Respuesta esperada: Una sola de error (un solo número con el botón de confirmación).

Esto afecta a los Bugs que hay que dividir.

Formularios

Formulario en el que se van validando todos los campos previamente:

No necesito un paso en mi caso de prueba que sea: El usuario Salva hace click en el botón "Aceptar".

Formularios en los que se valida campo por campo: nos dan la cantidad específica de casos de prueba.

Formularios que validan campo por campo en los pasos podemos poner muchos pasos que fallan con muchas clases de entradas inválidas y clases de salida inválida y se prueba todo junto. No haría falta hacer un caso de prueba por cada entrada.

Formulario en el que se validan todos los campos recién cuando doy al botón "Aceptar":

Necesito un paso en mi caso de prueba que sea: El usuario Salva hace click en el botón "Aceptar".

Solo un resultado esperado.

Tengo que hacer un caso de prueba por cada equivalencia inválida porque no tengo cómo saber qué se validó y qué no.

Ingresa y selecciona

NO ES LO MISMO

Ingresa: Implica que yo voy a escribir y las posibilidades de escribir algo mal son diferentes.

Seleccionar: Elijo de una lista que ya está disponible.

Selecciona:

Ejemplo: hacer click en la barra y scroll.

En seleccionar no sé cómo está programado.

No puedo asegurar cómo lo va a programar el desarrollador ni como lo va a hacer la persona.

Con las actividades tampoco puedo asegurar cómo lo va a hacer el desarrollador, backend, validaciones, auto complete, etc.

Ejemplo frontend un select pero probando el back con postman mando una solicitud e igual se tiene que validar.

Por lo tanto, además del "No seleccionó valor", me conviene agregar igual la clase de equivalencia "Poner un valor inválido" por más de que sea un select.

Listado

"Se debe mostrar un listado".

Por más que interactuemos después con él, ¿es una clase de salida? Sí.

Con esto validamos que no nos olvidamos de ninguna clase de salida.

Ejemplo listado de jugadores.

Kanban

Kanban:

Para procesos.

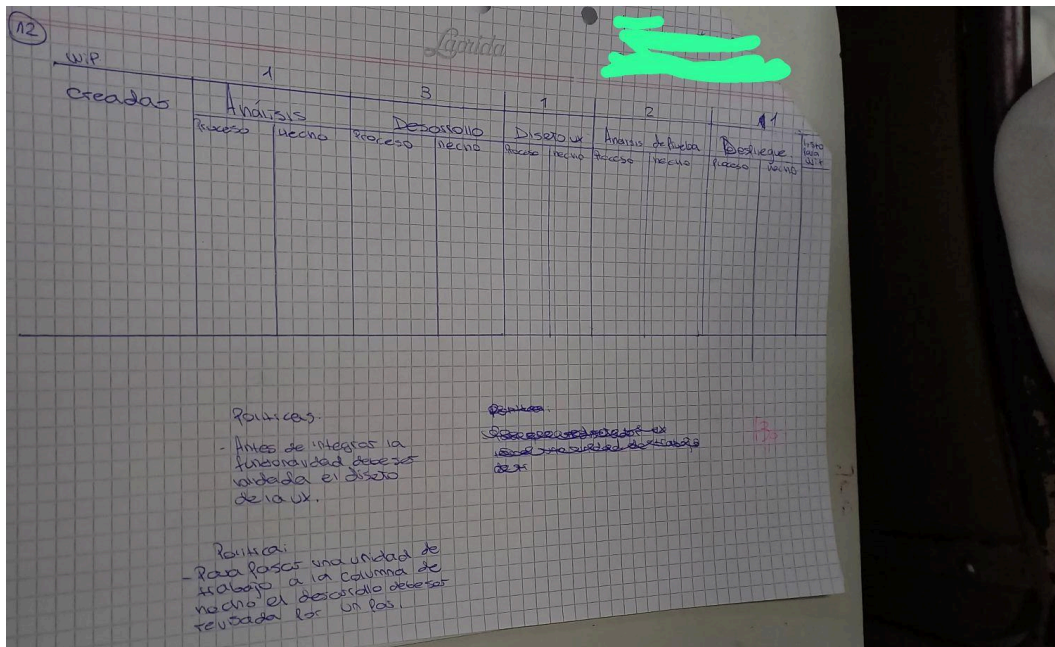
Para la gestión del cambio.

Tableros Kanban:

Modelamos tableros Kanban para dar visibilidad al proceso.

Una columna por cada actividad de nuestro proceso dividida en dos:

- Trabajo
- Acumulación



Columnas de hecho:

Columnas de hecho no tiene que ver con la cantidad de personas que tengo disponible para ejecutar las actividades.

Tiene que ver con permitir el pull de actividades.

WIP:

El WIP se va definiendo a partir de la prueba y el error.

Planteo una hipótesis, pruebo, ejecuto y hago un análisis para ver qué cambiar.

Busco que el trabajo fluya y para esto cada una de las columnas tiene que tener cierta holgura y movilidad para que un recurso se pueda mover a otra columna.

Evitar más demanda que oferta, ejemplo de carretera congestionada.

En realidad es bastante libre y lo decide cada equipo.

En general como ayuda par formular la primera hipótesis (no es una regla) el $WIP = 2n - 1$

n = cantidad de personas que yo tengo.

Ejemplo 3 desarrolladores -> $WIP = 2n - 1 = 2 \cdot 3 - 1 = 5$

EL WIP ES DE LAS COLUMNAS "IN PROGRESS".

NO TENGO WIP PARA LAS COLUMNAS DE ACUMULACIÓN.

TDD (Test-Driven Development) y ATDD (Acceptance Test-Driven Development)		
Criterio	TDD (Test-Driven Development)	ATDD (Acceptance Test-Driven Development)
Significado	Desarrollo guiado por pruebas.	Desarrollo guiado por pruebas de aceptación.
Enfoque principal	Validar el comportamiento técnico del código (unidad o componentes).	Validar que el sistema cumple los requisitos del usuario o negocio.
Nivel de prueba	Pruebas unitarias (nivel de código).	Pruebas de aceptación o funcionales (nivel de sistema).
Quién participa	Principalmente desarrolladores.	Desarrolladores, testers y clientes/usuarios colaboran.
Cuándo se escriben las pruebas	Antes de escribir el código de producción.	Antes de implementar una funcionalidad completa, a partir de criterios de aceptación.
Objetivo	Asegurar que el código funciona constantemente y es mantenible.	Asegurar que la funcionalidad satisface los requisitos del negocio.
Tipo de lenguaje usado en pruebas	Lenguaje de programación (JUnit, NUnit, etc.)	Lenguaje más orientado para negocio (Gherkin, Cucumber, etc.)
Ejemplo típico	"El método suma()" devuelve la suma correcta de dos números.	"Como usuario, quiero poder iniciar sesión para acceder a mi cuenta".
Granularidad	Muy fina (pruebas de métodos o clases).	Más amplia (pruebas de casos de uso o historias de usuario).
Ventaja principal	Mejora la calidad del código y facilita refactorizaciones.	Mejora la comunicación y asegura que se construye lo que el usuario necesita.
Resultado final	Código limpio, probado y libre de errores técnicos.	Funcionalidades completas verificadas con criterios de aceptación.
Herramientas comunes	JUnit, NUnit, xUnit, pytest.	Cucumber, FitNesse, Behave, SpecFlow.
Relación entre ambos	TDD se centra en cómo funciona el código internamente.	ATDD se centra en qué debe hacer el sistema desde la perspectiva del usuario.

Nexus vs. LeSS vs. LeSS Huge			
Framework	Nexus	LeSS	LeSS Huge
Aplicabilidad	De 3 a 9 equipos Scrum.	De 2 a 8 equipos.	Más de 8 equipos.
Miembros y Responsabilidades	1. Nexus Integration Team: Responsable de asegurar un incremento integrado. Compuesto por miembros de los Scrum Teams. 2. Product Owner (PO): Un solo PO para el producto. Responsable de maximizar el valor y gestionar el Product Backlog. 3. Scrum Master (SM): Responsable de que Nexus se entienda. Puede ser SM en uno o más equipos del Nexus. 4. Uno o más miembros del Nexus Integration Team.	1. Product Owner (PO): Un solo PO. 2. Scrum Master (SM): Un SM por línea de 1 a 3 equipos. Es un rol a tiempo completo. 3. Equipos: Autoorganizados, multifuncionales. Responsables de coordinar entre ellos.	Igual a LeSS. Un Product Manager (PO), un DdD, un DdR. 4. Area Product Manager. 5. Area Product Owners.
Eventos	1. Nexus Sprint: Produce un solo incremento integrado. 2. Refinamiento entre equipos: Evento para reducir dependencias e identificar qué equipo hará qué. 3. Nexus Sprint Planning: Coordina actividades de todos los equipos. 4. Nexus Daily Scrum: Asisten representantes de cada equipo. Se centra en problemas de integración. 5. Nexus Sprint Review: Revisamos las revisiones individuales. 6. Nexus Sprint Retrospective: Para planificar mejoras a nivel de Nexus. Se complementa con retros individuales.	1. Sprint Planning One: PO + Equipos (o repa) seleccionan items. 2. Sprint Planning Two: Los equipos deciden cómo harán los items y crean su Sprint Backlog. 3. Daily Scrum: Cada equipo tiene la suya. 4. Sprint Review: Una sola review para todo el producto, común a todos los equipos. 5. Retrospective: Cada equipo tiene la suya. 6. Overall Retrospective: Evento nuevo para discutir problemas entre equipos y organizacionales. 7. Product Backlog Refinement: Se realiza con todo el equipo. 8. LeSS Sprint: Un solo Sprint a nivel de producto: todos los equipos empujan y terminan a la vez.	Igual a LeSS. Un Sprint común para todos los equipos.
Artefactos	1. Product Backlog: Un solo Product Backlog. 2. Nexus Sprint Backlog: Compuesto por el Nexus Sprint Goal + Items de los equipos. Compromiso: Nexus Sprint Goal. 3. Integrated Increment: Suma de trabajo integrado. Compromiso: Definition of Done (DoD).	1. Incremento de producto: Salida del Sprint, integra el trabajo de todos los equipos. Un definición de Done (DoD) común para todos los equipos. 2. Product Backlog: Un solo PB para todos los equipos. 3. Sprint Backlog: Cada equipo tiene su propio Sprint Backlog.	Igual a LeSS. Un Product Backlog para todos, un DdD, un incremento. Diferencia a LeSS (Nexus): 5. Area Product Backlogs. 6. Area Product Vision.
Filosofía / Gestión	Tradicional (Cascada/Secuencial)	Máticas	Lean (Kanban)
Foco	En los 3 dominios: proceso, producto y producto (para mí solo proyecto y producto). Basado en la gestión de proyectos definidos.	En el producto. El "software funcionando" es la métrica principal. Las métricas son para el equipo y no son extrínsecas.	En el proceso. Mide la mejora de un flujo de trabajo continuo.
Métricas claves	1. Esfuerzo -> Proyecto 2. Tiempo -> Proyecto 3. Capacidad -> Producto 4. Defectos -> Producto 5. Costos 6. Riesgos	1. Velocidad (Velocity) -> Producto: La más importante. Mide Puntos de Historia (SP) aceptados por el PO en un Sprint. Se calcula. 2. Capacidad (Capacity) -> Producto: Métrica de proyecto. Mide el compromiso del equipo para un sprint (en horas o SPI). Se estima al inicio. 3. Running Tested Features (RTF) -> Producto: Mide la cantidad (absoluta) de features testeadas y funcionando.	1. Lead Time (Tiempo de Entrega) -> Proceso: Perspectiva del cliente. Mide desde que el cliente pide algo hasta que se lo entrega. 2. Cycle Time (Tiempo de Ciclo) -> Proceso: Vista interna. Mide desde que el equipo empieza a trabajar en algo hasta que lo entrega. 3. Touch Time (Tiempo de Toque) -> Proceso: Mide el tiempo efectivo de trabajo, sin incluir esperas. 4. Efectancia del ciclo de proceso: (Touch Time / Lead Time) -> Proceso.

Testing		
Filosofía / Gestión	Tradicional (Cascada/Secuencial)	Ágil (Iterativo/Incremental)
Momento de pruebas	Las pruebas se realizan al final del ciclo de vida, cuando se entrega el producto.	El testing se realiza por iteraciones.
Otros	(No aplica)	Esta aproximación puede introducir errores por la integración de los sucesivos incrementos?
Diferencias entre Scrum y Kanban		
Diferencia	Scrum	Kanban
Tiempo	Las iteraciones son de tiempo fijo.	El tiempo fijo es opcional. La cadencia puede variar. Pueden estar marcadas por la previsión de los eventos en lugar de tener un tiempo prefijado.
Compromiso	El equipo asume un compromiso de trabajo por iteración.	El compromiso es opcional.
Métrica por defecto	Para la planificación y mejora es la velocidad.	Por defecto es el Lead Time (tiempo de entrega promedio).
Reglas	Multifuncionales.	Multifuncionales o especializadas.
Diagramas de seguimiento	Deben emplearse gráficos Burndown.	No se prescriben diagramas de seguimiento.
Limitación WIP (Work in Progress)	Es indirecta y está marcada por el sprint.	Es directa para cada etapa de trabajo o estado del trabajo.
Agrupar trabajo	No se puede agrupar alcance una vez comenzado el sprint.	Siempre que haya capacidad disponible, se puede agrupar trabajo.
Compartido entre equipos	Cada equipo tiene su sprint backlog.	La estimación es opcional.
Permanencia del tablero	En cada sprint se limpia.	Todos los equipos comparten la misma pizarra o tabla Kanban.
Roles	Se prescriben tres roles: PO (Product Owner), SM (Scrum Master) y Equipo.	El tablero es persistente.
Priorización	El product backlog debe estar priorizado.	No se prescriben roles.
Funcionalidades	Se espera que las funcionalidades se dividan tal que se completen en un sprint.	La priorización es opcional.
		No se prescribe el tamaño de la funcionalidad.

Análisis de los principios Lean y explicación de las prácticas que propone Kanban para aplicar en forma concreta los principios	
1. Eliminar desperdicios	Visualizar el trabajo: permite visualizar el flujo de trabajo y exponer aquellas actividades que no aportan valor y aquellas que generan tiempos de espera que "traban" el flujo de trabajo.
2. Amplificar el aprendizaje	Limitar el WIP: el trabajo a medias genera retrabajo, lo cual es un desperdicio. Al limitar el WIP se evita ese trabajo a medias para concentrar esfuerzos.
3. Hacer explícitas las políticas	Hacer explícitas las políticas: que todos los miembros tengan acceso a las políticas usadas para cada unidad de trabajo, actividades a realizar, información necesaria del dominio, etc.
4. Posponer decisiones hasta el momento responsable	Evaluación y mejora continua: no introducir cambios grandes, sino que visualizar el proceso de trabajo actual y aplicar mejoras en él.
5. Ver el todo	Embeber la integridad conceptual
6. Dar poder al equipo	Limitar el WIP: particularmente el WIP en la columna backlog.
7. Mejorar colaborativamente	
8. Entregar lo antes posible	Visualizar el trabajo: a través del tablero permite que todos los miembros observen todo el trabajo que se hace, el sentido de cada una de las tareas y cómo estas conforman el flujo de trabajo como un todo que es esencial para la satisfacción de los clientes.
	Hacer explícitas las políticas y conformando un sistema de trabajo pull, donde cada miembro suma el trabajo cuando está en condiciones de realizarlo, y no se lo impone nadie.
	Gestionar el flujo de trabajo: permite ver de principio a fin los pasos que se deben seguir para lograr cumplir con las necesidades del cliente y hacer aquellos ajustes que sean necesarios para entregar el mayor valor posible, en el menor tiempo posible.

Cuadro comparativo Auditorías vs. Revisiones Técnicas		
Aspecto	Auditoría	Revisión
Para qué sirven	Para revisar qué se está haciendo en la realidad, contrastado con lo que se le propone a hacer.	Revisar errores en un análisis lo antes posible, para estar lo más preparados a hacer.
Quién	Auditor: auditado y gerente de calidad.	Auditor, moderador, auditado, lector, inspector.
Beneficios	Opiniones independientes, identificar áreas de mejoramiento, asegurar que se cumplen expectativas, dar visibilidad a procesos de trabajo, etc.	Detección de errores, evitar el desperdicio de recursos, asegurar que se cumplen expectativas, dar visibilidad a procesos de trabajo, etc.
Etapas	1. Planificación 2. Desarrollo 3. Análisis y Recuento de resultados 4. Seguimiento	En revisiones informales no hay pasos a seguir. En revisiones formales: 1. Planificación 2. Visión general 3. Presentación 4. Revisión de información 5. Corrección 6. Seguimiento
Tipo	De proyecto, de calidad, de producto (fácil o funcional).	Formales e informales.

100

Cuadro comparativo CMMI por Etapas y Continuo		
Aspecto	Por etapas	Continuo
Características	Análisis determinado nivel de madurez de una organización como un todo, dependiendo de qué tanto creemos madurez, de acuerdo a la especificación en el modelo CMMI.	Análisis determinado nivel de madurez de un área de proceso de una organización, dependiendo de qué tanto creemos madurez, considerando a esa área, de acuerdo a lo que la organización quiere.
Ventajas	Permite integrar las formas de trabajo de toda la organización. Identifica cómo nivel de calidad en toda la organización, según el nivel que se quiere. Permite saber bases en la organización y entender los niveles de calidad en toda la empresa.	Permite a una organización especializarse en un área de proceso, independientemente del resto de áreas. La organización sigue sus prácticas reales. De acuerdo al área en la que se especializa.
Desventajas	Se deben realizar ciertos actividades en áreas de proceso que quizás no sean relevantes para la empresa. La organización como un todo se sigue considerando, independientemente de que no requieren las actividades que plantea CMMI para el área de proceso.	Puede quedar áreas de proceso sin demasiada calidad, debido a que se centran los esfuerzos en una zona. La organización como un todo se sigue considerando, independientemente de que no requieren las actividades que plantea CMMI para el área de proceso.
Diferencias	- Modelo 1 a 5 - Niveles: indican madurez organizacional - Definidos por un conjunto de áreas de proceso - Permite una instancia para mejorar determinadas áreas de proceso	- Modelo 0 a 5 - Niveles: indican capacidad de un área de proceso - Definidos por cada área de proceso - Permite mejorar en un área de proceso que se desea

hallazgos	Buenas prácticas, observaciones y evidencias	Errores

Cuadro comparativo Gestión Tradicional vs. Gestión Ágil		
Aspecto a comparar	Gestión Tradicional	Gestión Ágil
Requisitos a cambios	Poco flexibles	Altamente flexibles
Requisitos en el desarrollo del proyecto	Agg -> se define en etapas de desarrollo	Agg -> se define cuando sea necesario
Tiempo de entrega de software (SI)	Variable	Temperano
Retransmisión del cliente para incluir nuevos requerimientos	Rara	Alta
Participaciones en estimaciones	Un hace el líder de proyecto	Los hace el equipo de desarrolladores
Confirmación de equipo	Indirecta, con roles definidos	Autogestionado según fortalezas de cada miembro
Proceso de desarrollo	Definido	Empírico

100

Códo de vida admitido	Cualquiera	Restricto e incremental
Se estima	Recursos y tiempos necesarios para el desarrollo	Alumnos a implementar en iteraciones (Cascada)
Se comienta	Los requisitos identificados en etapas de requerimientos	El equipo, recursos y tiempo de trabajo
Planificación de tareas	Estimados con detalle al planificar	Estimados con detalle al planificar
Preguntas frecuentes de la ingeniería de software (Sumariadas)		
¿Cuál es software?	Programas de cómputo y documentación asociados. Los productos de software se desarrollan para su cliente en particular y para un mercado en general.	
¿Cuáles son los atributos de buen software?	El buen software debe entregar al usuario la funcionalidad y el desempeño requeridos, y debe ser sustentable, confiable y útil.	
¿Cuál es ingeniería de software?	La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.	
¿Cuáles son las actividades fundamentales de la ingeniería de software?	Especificación, desarrollo, verificación y evaluación del software.	
¿Cuáles son las principales metas que enfrenta la ingeniería de software?	Se enfrentan con una cantidad creciente, demandando por tiempos de desarrollo más cortos y desarrollo de software confiable.	

100

¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?	Aun cuando todos los proyectos de software deben gestionarse y desarrollarse de manera profesional, existen diferencias de manera adecuada para distintos tipos de sistema. Por ejemplo, los juegos comerciales deben cumplir con una serie de requisitos, mientras que los sistemas críticos de medicina de seguridad requieren de una especificación completa y detallada para su desarrollo. Por lo tanto, no puede decirse que un método sea mejor que otro.	